INTERNAL PROJECT DOCUMENTATION

AI Document Auditor

An asynchronous, citation-backed RAG system for auditing legal, financial, and employment documents — architecture, rationale, and anti-hallucination design.

Prepared for: Harrison Ekpobimi

Date: September 2026

Scope: Backend (Python / FastAPI), Frontend (React / TypeScript), CLI

Contents

1. What This Project Is
2. The Business Problem
3. Architecture
4. Why This Architecture
5. Technology Stack
6. User Journeys
7. Guarding Against Hallucination
8. Current Status & Known Limitations

01

What This Project Is

The AI Document Auditor turns an unstructured document — a contract, a financial report, an employment agreement — into a structured, evidence-backed compliance audit. Instead of a chat window where you ask a model questions and hope the answer is right, it runs a fixed or custom **checklist** of questions against the document and returns, for every item, a verdict that is required to carry its own proof.

Every result is one of exactly four outcomes:

PASSED — the document satisfies this check

FAILED — a confirmed violation was found

CONTRADICTORY — the document disagrees with itself

NOT_FOUND — the concept simply isn't in the document

Every non-`NOT_FOUND` result must include a **verbatim quotation** from the source text and the **exact page number** it came from — not a paraphrase, not a summary. A human reviewer can open the document to that page and check the claim in seconds.

The system ships as three things built on the same core engine: a one-shot **CLI** for scripted or local use, an **HTTP API** with job-based polling, and a **React web app** for interactively uploading a document, picking a checklist, and reviewing results.

02

The Business Problem

Manual document review doesn't scale

Legal, financial, and HR teams routinely need to check documents — contracts, reports, agreements — against a standard checklist: is there a liability cap, does the governing law create a conflict, does a non-compete term stay within a reasonable limit. Doing this by hand is slow, repetitive, and the kind of task where a tired reviewer misses something a machine wouldn't — especially **internal contradictions** buried pages apart in a long document.

Naive LLM document Q&A isn't trustworthy enough to replace it

The obvious fix — "ask an LLM about the document" — has a well-known failure mode: it hallucinates. It can state a fact that isn't in the source, invent a citation, or paraphrase a clause into something subtly wrong. In a compliance or legal context, that isn't a minor UX flaw; a confidently wrong answer is worse than no answer, because it's the kind of error a reviewer is likely to trust and not double-check.

What was actually needed

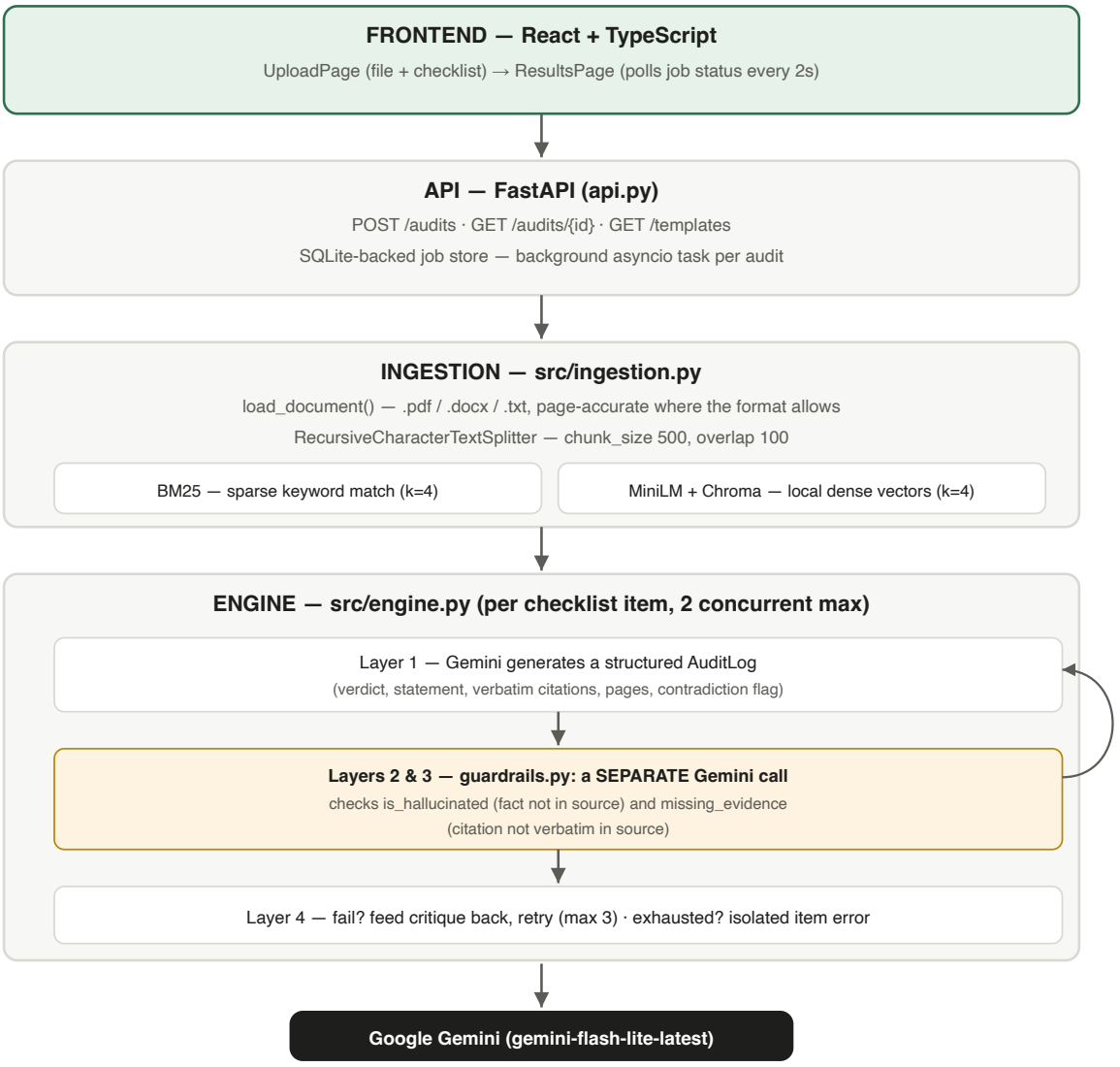
- Speed — audit a document against a full checklist in one pass, not one manual re-read per question.
- Evidence, not assertions — every finding traceable to an exact quote and page.
- Honesty about absence — if a checklist item doesn't apply to this document, say so, don't invent an answer to look complete.
- Contradiction detection — actively compare fragments across the whole document, not just answer each question in isolation.
- A workflow that fits how the work is actually organized — different checklists for different document types (a contract review isn't the same checklist as a financial report review).

> **IN ONE LINE**
>
> This project exists to make LLM-based document review fast *and* trustworthy at the same time — by refusing to treat "trustworthy" as optional.

03

Architecture

The system is five layers, each with one clear job:



Request lifecycle, start to finish

1. User uploads a file and picks a checklist template (or writes a custom one) in the web app, which `POST`s to `/audits`.
2. The API resolves the checklist, saves the upload, creates a job in `queued` state, and starts processing as a background task — the response returns immediately with a job ID.
3. The job loads and chunks the document, builds one hybrid retriever, then runs every checklist question through the engine, throttled to two at a time.
4. The frontend polls the job every two seconds; each checklist item renders independently the moment it lands — `queued` → `done` or `error` — instead of waiting for the whole batch.
5. Once every item resolves, the job completes and the full aggregated report becomes available for review or export.

04

Why This Architecture

Every non-obvious decision below came from something that was actually observed breaking, not from following a default. That's worth stating plainly: several of these choices exist specifically because the more obvious first attempt failed in a way that was measured and fixed, not guessed at.

Hybrid retrieval (BM25 + vector), not vector search alone

Pure semantic search can miss an exact phrase or a specific section reference that doesn't carry much "semantic" weight but matters enormously for a legal document. BM25 catches literal keyword/phrase overlap; the vector side catches conceptual matches the same words don't appear in. Combined with equal weighting, the system gets both.

Local embeddings (MiniLM), not Gemini's embedding API

Ingestion was originally wired to Gemini's embedding endpoint. It was replaced with a local `sentence-transformers/all-MiniLM-L6-v2` model after a direct, controlled comparison — the same document, the same checklist, run once against each — came back with **identical verdicts on all four outcome types** (a genuine contradiction, a pass, a fail, and a not-found). Local embeddings remove ingestion's dependency on network access, an API key, or Gemini's quota entirely, at no measured cost to result quality on the case that mattered most (a contradiction that depends on correct retrieval).

A job-polling API, not a synchronous request/response

A full audit is not fast — a multi-item checklist with retries can take well over a minute, and free-tier rate limits make that worse under load. A synchronous endpoint would either time out or force the frontend to block with no feedback. The job model lets the UI show real per-item progress and lets one item's failure be isolated rather than taking down the whole request.

Larger chunk overlap and a sentence-aware splitter

> **WHAT ACTUALLY HAPPENED**
>
> With the original 50-character overlap, a single sentence stating a document's governing law was split exactly across a chunk boundary — the retriever picked up the chunk with the setup clause but not the chunk containing "...shall be governed exclusively by the laws of the United Kingdom," and the audit silently missed a real contradiction. Increasing the overlap to 100 characters and adding a sentence-boundary separator (`". "`) to the splitter fixed it — confirmed by rerunning the same case and getting the correct result four times in a row.

Retrieving four chunks per query, not two

Fixing the chunk boundary wasn't sufficient on its own — even with the sentence intact in a single chunk, a top-2 retrieval sometimes still failed to surface it, because a different, more keyword-relevant chunk out-

ranked it. Widening retrieval to the top four candidates on both the keyword and vector side closed that gap. Both fixes were necessary; neither alone was.

A second, independent model checking the first one's work

Asking one model to both generate a finding and grade its own accuracy is a weak check — it's evaluating a claim it just made itself. Running a completely separate adversarial call against the same evidence catches errors a single pass wouldn't flag on its own. See Section 7 for the full design.

TypeScript on the frontend, not plain JavaScript

The backend enforces its data contracts everywhere with Pydantic — a shape mismatch fails loudly by design. The frontend was originally plain JavaScript, which meant that same API response could be consumed with zero type checking. The migration closed that gap: citation and page-number arrays are paired positionally (an easy thing to get wrong by hand), and the verdict field is a closed set of four values — both are exactly the kind of contract a type system is built to protect.

Checklist templates as structured data, kept out of the LLM's schema

Each checklist item has an internal `id`, a short display `label`, and the actual `question` sent to the model. The `id` / `label` are deliberately never passed to the LLM or included in its structured-output schema — the model only ever sees the plain question text. That keeps the audit engine fully decoupled from presentation/template concerns, and means the model has one less thing it could get wrong or echo back incorrectly.

05

Technology Stack

Backend

LAYER	TECHNOLOGY	ROLE
Language / runtime	Python 3.11, asyncio	Concurrent, throttled checklist processing
API framework	FastAPI + Uvicorn	Job-based HTTP API behind the frontend
Orchestration	LangChain (core, community, text-splitters)	Retrievers, prompt templates, structured output binding
Schema enforcement	Pydantic v2	Every LLM output and API payload is a validated model, not free text
Generation & judging	Google Gemini — `gemini-flash-lite-latest`	Both the audit model and the independent guardrail judge
Vector store	ChromaDB	Local embedded vector index, rebuilt per run
Embeddings	sentence-transformers (`all-MiniLM-L6-v2`)	Local, no network dependency for ingestion
Keyword retrieval	rank_bm25	Sparse side of the hybrid retriever
Document parsing	pypdf, python-docx	Per-format ingestion with real or best-effort page accuracy
Testing	pytest, pytest-asyncio	Unit and regression coverage on the engine and ingestion layers

Frontend

LAYER	TECHNOLOGY	ROLE
Language	TypeScript	Typed API contracts mirroring the backend's Pydantic schemas
UI framework	React 19	Component-based SPA
Build tool	Vite	Dev server + production bundling
Routing	React Router	Upload page ↔ results page, with real shareable URLs
Linting	oxlint	Fast Rust-based linting

Why Gemini specifically

The generation and guardrail models both run on `gemini-flash-lite-latest` rather than a larger model — chosen after directly hitting free-tier rate limits on both `gemini-2.5-pro` (zero available quota) and the full `gemini-flash-latest` (5 requests/minute, too tight for a multi-item checklist). The lite tier had materially more usable headroom and, combined with a concurrency throttle of two simultaneous items, has run reliably since.

06

User Journeys

Web application

1. **Upload**
   Drag in a .pdf, .docx, or .txt file — type is validated client-side before it ever reaches the server.

2. **Choose a checklist**
   Pick General Contract, M&A, Financial Report, or Employment Contract — each previews its questions before you commit — or switch to Custom and write your own, one per line.

3. **Watch it run**
   Each checklist item updates independently as it completes — a progress banner shows "N of M checks complete" rather than one blocking spinner.

4. **Review results**
   Every item shows its verdict badge, plain-language finding, and an expandable list of verbatim citations paired with page numbers. Contradictions get a highlighted explanation.

5. **Triage and act**
   A health score and a risk summary surface what needs attention first; clicking a risk item jumps straight to its card. Export the full report as Markdown, or re-run the audit as a fresh job.

Command line

1. **Run with or without arguments**
   `python main.py contract.pdf --template mna_contract`, or just `python main.py` to be prompted interactively for a document path.

2. **Pipeline runs end-to-end**
   Ingestion, retrieval, and the full checklist run synchronously in the terminal, with progress printed as it goes.

3. **Structured JSON output**
   The full report prints as strictly-typed JSON — suitable for piping into another tool or script.

A real result, exactly as produced

CHECKLIST ITEM: "IDENTIFY THE GOVERNING LAW STATE AND VERIFY IF CONFLICTS EXIST."

CONTRADICTORY

The document contains a direct contradiction regarding the governing law. Section 14 designates the State of New York, while Section 22.9 designates the United Kingdom.

> *"...shall be governed exclusively by...the laws of the State of New York" — page 2*

> *"...shall be governed exclusively by the laws of the United Kingdom" — page 5*

07

Guarding Against Hallucination

This is the part of the system the rest of the architecture exists to serve. The defense is layered — no single mechanism is trusted alone.

1. Structural constraint: no verdict without evidence

The audit model doesn't produce free text — it fills out a strict schema (`AuditLog`) where verbatim citations and page numbers are **required fields**, not optional context. It is structurally impossible to return a finding without also committing to exactly where in the document it came from.

2. A second, independent model grades the first one's work

A separate Gemini call — explicitly prompted as an adversarial QA judge — re-reads the raw source text against the generated statement and citations, and checks two specific things: whether the statement contains any fact not explicitly present in the source, and whether the claimed citations exist **word-for-word** in the source. The model that wrote the finding never grades its own homework.

3. Failure triggers a targeted retry, not a silent failure

If the judge flags a problem, its specific critique is fed back into the next attempt's prompt and the model tries again — up to three attempts. If all three fail, that single checklist item is isolated as an error with the real reason attached; it does not take down the rest of the audit.

4. Retrieval quality, upstream of generation entirely

A guardrail can only catch a hallucination after the fact. The other half of the defense is making sure the model has the right evidence in front of it before it ever has to guess — the chunk-overlap and retrieval-width fixes in Section 4 exist specifically because incomplete context is exactly when a model is most tempted to fill a gap on its own.

> **PROOF — HONEST ABSENCE OVER FABRICATION**
>
> Running a 12-item M&A checklist against a document that is actually an investment advisory agreement, not an M&A contract, nine items correctly returned **NOT_FOUND** — no purchase price, no closing date, no conditions precedent exist in this document type, and the system said so instead of inventing plausible-sounding values to look complete.

> **PROOF — THE GUARDRAIL CATCHING A REAL OVER-EXTRAPOLATION, LIVE**
>
> In production use, a non-compete check failed because the audit model wrote that a 24-month restriction "exceeds the twelve (12) month maximum limit" — but that 12-month threshold came from the checklist question itself, not the source document. The guardrail correctly flagged this as an external judgment not grounded in the raw text, and the retry loop engaged exactly as designed.

An honest limitation

WORTH STATING PLAINLY

The guardrail is itself an LLM call, not a deterministic verifier — it is a second opinion, not a mathematical guarantee. It can in principle be wrong in either direction: too lenient, missing a real hallucination, or too strict, flagging a defensible inference as fabrication. This system meaningfully reduces hallucination risk and forces every claim to carry checkable evidence — it does not claim to eliminate the risk entirely.

08

Current Status & Known Limitations

Shipped and in active use today:

- Multi-format ingestion (.pdf / .docx / .txt) with page-accurate citations where the format supports it.
- Four checklist templates, plus fully custom checklists.
- The complete four-layer anti-hallucination pipeline.
- A React + TypeScript web app: upload, live per-item progress, health score, risk summary, Markdown export, re-run.
- A job-based HTTP API and a standalone CLI, sharing the same core engine.
- Tier 0 document viewing — `GET /audits/{id}/document` serves the original upload back, and every PDF citation's page number links straight to it via the browser's native viewer (a `#page=N` fragment), no extra dependency required.

Known, deliberate limitations — not oversights:

- **Persistence is minimal, not multi-instance.** Jobs are written through to a local SQLite file and survive a server restart, but there's still no job-listing endpoint, and a job still running when the process dies is marked failed rather than resumed. One file, one process — not shared across replicas.
- **Authentication is a single shared key, not accounts.** One `API_KEY` gates the whole API; there's no per-user identity or ability to revoke one caller without rotating the key for everyone.
- **No citation highlighting.** The document viewer (above) jumps to the right page; it doesn't highlight where on that page the cited text sits. That needs bounding-box coordinates the ingestion layer doesn't extract today — `pypdf`'s text extraction returns flat text, no layout data — and a client-side text-search alternative would inherit the same extraction fragility (broken mid-word line-wraps, whitespace mismatches) seen elsewhere in this project. `.docx` / `.txt` have no viewer at all, only download.
- **Audit only, not an editor.** The system reports findings; it does not propose or apply changes to a document.
- **Free-tier rate limits are real.** Concurrency is deliberately throttled to two simultaneous checklist items to stay within Gemini's free-tier quota.

> **ROADMAP, IN PRIORITY ORDER**
>
> A job-listing endpoint on top of the persistence now in place; a freeform "ask a question" capability reusing the existing retriever, once a real reason to prioritize it over the above emerges; true citation highlighting on top of the Tier 0 viewer shipped above, scoped as its own project given the ingestion rework it requires; and, only once multiple distinct users are an actual requirement rather than a hypothetical, real per-user accounts on top of today's single shared API key.